

Práctica: Criptografía cuántica – Protocolo BB84

1. Implementación sin atacante

Proporcionar una implementación base del protocolo BB84 sin presencia de atacante.

Se debe tener instalado el SDK de código abierto creado por IBM Qiskit en sus versiones `qiskit==2.3.1` y `qiskit-aer==0.17.2`.

1.1 Librerías y variables iniciales

```
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator

n = 10 ** 3
qubit_index = 0
backend = AerSimulator()
message = [QuantumCircuit(1, 2) for _ in range(n)]
```

1.1 Preparación de estados (Alice)

Alice genera dos secuencias aleatorias: una de bits y otra de bases. Cada bit se codifica en un cúbit dependiendo de la base seleccionada (computacional o Hadamard).

```
alice_bits = np.random.randint(2, size=n)
alice_bases = np.random.randint(2, size=n)

for i, qc in enumerate(message):
    if alice_bits[i] == 1:
        qc.x(0)
    if alice_bases[i] == 1:
        qc.h(0)
```

1.2 Cambio de bases (Bob)

Bob selecciona de forma aleatoria una base de medida para cada cúbit recibido.

```
bob_bases = np.random.randint(2, size=n)
```



```
for i, qc in enumerate(message):  
    if bob_bases[i] == 1:  
        qc.h(0)
```

1.3 Medida

Posteriormente, realiza la medición en dicha base.

```
classical_bit_index_for_bob_measurement = 0  
  
for qc in message:  
    qc.measure(qubit_index, \  
               classical_bit_index_for_bob_measurement)
```

1.4 Comparación de bases

Alice y Bob comparan públicamente las bases utilizadas y conservan únicamente aquellos bits en los que ambas coinciden.

```
cbit = classical_bit_index_for_bob_measurement  
bob_measured_bits = []  
for qc in message:  
    transpiled_qc = transpile(qc, backend)  
    result = backend.run(qc, shots=1).result()  
    counts = result.get_counts()  
    measured_bit = list(counts.keys())[0][::-1][cbit]  
    bob_measured_bits.append(measured_bit)  
  
alice_key = []  
bob_key = []  
for i in range(n):  
    if alice_bases[i] == bob_bases[i]:  
        alice_key.append(int(alice_bits[i]))  
        bob_key.append(int(bob_measured_bits[i]))
```

1.5 Cálculo del QBER

Calcular la tasa de error cuántico (QBER) como valor entre 0 y 1, comparando los bits medidos de Alice y Bob en las posiciones en las que las bases coinciden.